

MongoDB Student Management System (collegeDB)

Name	B. Venkata sai laxmi
Roll No.	A24126552074
Course	Full Stack Web Development (23CM4121)
Assignment No.	2

1. Title

Development of a MongoDB-based student information system to store, retrieve, update and manage student records, with indexing and a marks dashboard.

2. Aim

To create a MongoDB database named collegeDB with a students collection and perform insert, display, search, update, delete, sort and indexing operations, and to build a dashboard of real-time queries on the student data.

3. Objective

- Create the database collegeDB and the collection students.
- Store each student as a document with rollNo, name, branch, year, marks and email.
- Perform CRUD operations (Create, Read, Update, Delete) using mongosh.
- Filter, search and sort records using query operators.
- Create an index on rollNo and demonstrate how it speeds up searching.
- Build a dashboard using queries on marks, branch and ranking.

4. Theory

MongoDB is a NoSQL, document-oriented database. Data is stored as BSON (binary JSON) documents inside collections, and a group of collections forms a database. Unlike relational tables, documents in a collection do not need a fixed schema, which makes MongoDB flexible for web applications.

CRUD operations: insertOne() and insertMany() create documents; find() and findOne() read them; updateOne() and updateMany() modify them using operators such as \$set and \$inc; deleteOne() and deleteMany() remove them.

Query operators: \$gt, \$gte, \$lt and \$lte compare values; filters with several fields are combined with AND. sort({ marks: -1 }) orders results in descending order and limit(n) restricts the number of results.

Indexing: an index is a sorted data structure on one or more fields. Without an index MongoDB performs a collection scan (COLLSCAN) and reads every document. With an index on the searched field it performs an index scan (IXSCAN) and jumps directly to the matching entries. A unique index on rollNo additionally prevents duplicate roll numbers. The explain("executionStats") method shows which plan was used and how many documents were examined.

Aggregation: the aggregate() pipeline with \$group processes documents in stages, for example to calculate the count, average and maximum marks for each branch.

5. Algorithm

1. Start the MongoDB server and open mongosh.
2. Switch to the database collegeDB with use("collegeDB") and create the students collection.
3. Insert the student documents using insertOne() and insertMany().
4. Display all students, then filter by branch and by marks greater than 75.
5. Search a student by rollNo and search by marks and year conditions.
6. Update the marks of one student, then update the email and branch of other students.
7. Delete a student using rollNo.
8. Display students sorted in descending order of marks.
9. Insert dummy records, run explain() without an index, create an index on rollNo, run explain() again and compare.
10. Run the dashboard queries: marks above 80, marks below 50, highest scorer, branch filter, sorted list.
11. Stop.

6. Program

Commands are executed in mongosh. The complete scripts are in the folder assignment-2 (01_setup_insert.js to 05_dashboard_queries.js and dashboard.js).

6.1 Create database, collection and insert records

```
use("collegeDB")
db.createCollection("students")
```

```
db.students.insertOne({
  rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML",
  year: 3, marks: 85, email: "ravi@example.com"
})
```

```
db.students.insertMany([
  { rollNo: "23CM002", name: "Sneha Reddy", branch: "CSE-AIML", year: 3, marks: 92, email:
"sneha@example.com" },
  { rollNo: "23CM003", name: "Arjun Varma", branch: "CSE", year: 2, marks: 68, email:
"arjun@example.com" },
  { rollNo: "23CM004", name: "Priya Sharma", branch: "ECE", year: 4, marks: 74, email:
"priya@example.com" },
  { rollNo: "23CM005", name: "Karthik Rao", branch: "CSE", year: 3, marks: 45, email:
"karthik@example.com" },
  { rollNo: "23CM006", name: "Divya Teja", branch: "IT", year: 2, marks: 81, email:
"divya@example.com" },
  { rollNo: "23CM007", name: "Manoj Kumar", branch: "ECE", year: 1, marks: 39, email:
"manoj@example.com" },
  { rollNo: "23CM008", name: "Anitha Devi", branch: "CSE-AIML", year: 4, marks: 77, email:
"anitha@example.com" },
  { rollNo: "23CM009", name: "Rahul Verma", branch: "IT", year: 3, marks: 58, email:
"rahul@example.com" },
  { rollNo: "23CM010", name: "Lakshmi Prasad", branch: "CSE", year: 1, marks: 88, email:
"lakshmi@example.com" }
])
```

```
])
```

6.2 Display and search operations

```
db.students.find() // all students
db.students.find({ branch: "CSE-AIML" }) // by branch
db.students.find({ marks: { $gt: 75 } }) // marks > 75
db.students.findOne({ rollNo: "23CM001" }) // search by rollNo
db.students.find({ marks: { $gte: 60, $lte: 80 } }) // marks condition
db.students.find({ year: 3 }) // year condition
db.students.find({ year: 3, marks: { $gt: 70 } }) // combined condition
```

6.3 Update and delete operations

```
db.students.updateOne({ rollNo: "23CM001" }, { $set: { marks: 90 } })
db.students.updateOne({ rollNo: "23CM003" }, { $set: { email: "arjun.varma@college.edu" } })
db.students.updateOne({ rollNo: "23CM009" }, { $set: { branch: "CSE" } })
db.students.updateMany({ year: 1 }, { $inc: { marks: 5 } }) // bonus marks for year 1
db.students.deleteOne({ rollNo: "23CM007" })
```

6.4 Sorting and indexing

```
db.students.find({}, { _id: 0, rollNo: 1, name: 1, marks: 1 }).sort({ marks: -1 })

// 20,000 dummy records (rollNo "TEST00001" ... "TEST20000") are inserted for the demo
db.students.find({ rollNo: "TEST19999" }).explain("executionStats") // before index
db.students.createIndex({ rollNo: 1 }, { unique: true })
db.students.find({ rollNo: "TEST19999" }).explain("executionStats") // after index
db.students.getIndexes()
db.students.deleteMany({ rollNo: /^TEST/ }) // clean up
```

6.5 Real-time extension: dashboard queries

```
db.students.find({ marks: { $gt: 80 } }) // scoring above 80
db.students.find({ marks: { $lt: 50 } }) // scoring below 50
db.students.find().sort({ marks: -1 }).limit(1) // highest-scoring student
db.students.find({ branch: "CSE-AIML" }) // students of a branch
db.students.find().sort({ marks: -1 }) // sorted by marks

db.students.aggregate([ // extra: branch-wise summary
  { $group: { _id: "$branch", students: { $sum: 1 },
    avgMarks: { $avg: "$marks" }, maxMarks: { $max: "$marks" } } },
  { $sort: { avgMarks: -1 } }
])
```

7. Code Explanation

- **use("collegeDB")** selects the database; MongoDB creates it when the first document is stored.
- **createCollection("students")** creates the collection that holds the student documents.
- **insertOne()** / **insertMany()** insert a single document or an array of documents.

- **find(filter)** returns all documents matching the filter; find() with no filter returns everything.
- **findOne({ rollNo })** returns only the first matching document, used to search by roll number.
- **\$gt, \$gte, \$lt, \$lte** greater than, greater than or equal, less than, less than or equal.
- **\$set** changes only the named field of the matched document.
- **\$inc** increases a numeric field by the given value (used to add bonus marks).
- **updateOne() / updateMany()** update the first matching document or all matching documents.
- **deleteOne({ rollNo })** removes the first document matching the roll number.
- **sort({ marks: -1 })** sorts by marks in descending order (1 means ascending).
- **limit(1)** returns only one document, giving the highest scorer after sorting.
- **createIndex({ rollNo: 1 }, { unique: true })** builds an ascending index on rollNo and rejects duplicate roll numbers.
- **explain("executionStats")** shows the query plan (COLLSCAN or IXSCAN) and the number of documents examined.
- **aggregate([\$group, \$sort])** groups students by branch and calculates count, average and maximum marks.

8. Test Cases

No.	Operation	Query	Expected result
1	Insert records	insertOne + insertMany	10 documents stored
2	Display all	find()	All 10 students listed
3	Filter by branch	find({ branch: "CSE-AIML" })	23CM001, 23CM002, 23CM008
4	Marks above 75	find({ marks: { \$gt: 75 } })	5 students (marks 77 to 92)
5	Search by rollNo	findOne({ rollNo: "23CM001" })	Ravi Kumar's document
6	Condition search	marks 60 to 80; year 3	3 students; 4 students
7	Update marks	\$set marks: 90 for 23CM001	matchedCount 1, modifiedCount 1
8	Update email / branch	\$set for 23CM003 / 23CM009	modifiedCount 1 each
9	Delete	deleteOne({ rollNo: "23CM007" })	deletedCount 1, 9 students remain
10	Sort descending	sort({ marks: -1 })	93, 92, 90, 81, 77, 74, 68, 58, 45
11	Index demo	explain() before and after index	COLLSCAN, then IXSCAN with 1 document examined
12	Dashboard	marks > 80, < 50, top scorer	4 students; 1 student; Lakshmi Prasad (93)

9. Output

Results below are for the sample data in Section 6. Auto-generated `_id` values are omitted from the tables.

9.1 Insert records

insertOne

```
{ acknowledged: true, insertedId: ObjectId('...') }
```

insertMany

```
{ acknowledged: true,
  insertedIds: { '0': ObjectId('...'), '1': ObjectId('...'), ... '8': ObjectId('...') } }
```

db.students.countDocuments()

10

9.2 Display all students

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	85	ravi@example.com
23CM002	Sneha Reddy	CSE-AIML	3	92	sneha@example.com
23CM003	Arjun Varma	CSE	2	68	arjun@example.com
23CM004	Priya Sharma	ECE	4	74	priya@example.com
23CM005	Karthik Rao	CSE	3	45	karthik@example.com
23CM006	Divya Teja	IT	2	81	divya@example.com
23CM007	Manoj Kumar	ECE	1	39	manoj@example.com
23CM008	Anitha Devi	CSE-AIML	4	77	anitha@example.com
23CM009	Rahul Verma	IT	3	58	rahul@example.com
23CM010	Lakshmi Prasad	CSE	1	88	lakshmi@example.com

9.3 Students of branch CSE-AIML

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	85	ravi@example.com
23CM002	Sneha Reddy	CSE-AIML	3	92	sneha@example.com
23CM008	Anitha Devi	CSE-AIML	4	77	anitha@example.com

9.4 Students who scored more than 75

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	85	ravi@example.com
23CM002	Sneha Reddy	CSE-AIML	3	92	sneha@example.com
23CM006	Divya Teja	IT	2	81	divya@example.com
23CM008	Anitha Devi	CSE-AIML	4	77	anitha@example.com
23CM010	Lakshmi Prasad	CSE	1	88	lakshmi@example.com

9.5 Search by rollNo (23CM001)

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	85	ravi@example.com

9.6 Search by condition**marks between 60 and 80**

rollNo	name	branch	year	marks	email
23CM003	Arjun Varma	CSE	2	68	arjun@example.com

rollNo	name	branch	year	marks	email
23CM004	Priya Sharma	ECE	4	74	priya@example.com
23CM008	Anitha Devi	CSE-AIML	4	77	anitha@example.com

year = 3

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	85	ravi@example.com
23CM002	Sneha Reddy	CSE-AIML	3	92	sneha@example.com
23CM005	Karthik Rao	CSE	3	45	karthik@example.com
23CM009	Rahul Verma	IT	3	58	rahul@example.com

year = 3 and marks > 70

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	85	ravi@example.com
23CM002	Sneha Reddy	CSE-AIML	3	92	sneha@example.com

9.7 Update marks of a student (23CM001)

{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }

rollNo	name	branch	year	marks	email
23CM001	Ravi Kumar	CSE-AIML	3	90	ravi@example.com

9.8 Update email and branch

Email of 23CM003

{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }

rollNo	name	branch	year	marks	email
23CM003	Arjun Varma	CSE	2	68	arjun.varma@college.edu

Branch of 23CM009

{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }

rollNo	name	branch	year	marks	email
23CM009	Rahul Verma	CSE	3	58	rahul@example.com

Bonus: updateMany({ year: 1 }, { \$inc: { marks: 5 } })

{ acknowledged: true, insertedId: null, matchedCount: 2, modifiedCount: 2, upsertedCount: 0 }

9.9 Delete a student (23CM007)

{ acknowledged: true, deletedCount: 1 }

db.students.countDocuments()

9

9.10 Students in descending order of marks

rollNo	name	marks
23CM010	Lakshmi Prasad	93
23CM002	Sneha Reddy	92

rollNo	name	marks
23CM001	Ravi Kumar	90
23CM006	Divya Teja	81
23CM008	Anitha Devi	77
23CM004	Priya Sharma	74
23CM003	Arjun Varma	68
23CM009	Rahul Verma	58
23CM005	Karthik Rao	45

9.11 Index on rollNo and why it is useful

createIndex

rollNo_1

Searching rollNo = "TEST19999" in a collection of 20009 documents

Measure	Before index	After index
Winning plan stage	COLLSCAN	IXSCAN
Documents examined	20009	1
Index keys examined	0	1
Documents returned	1	1

db.students.getIndexes()

```
[
  { v: 2, key: { _id: 1 }, name: '_id_ ' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1', unique: true }
]
```

Clean-up: deleteMany({ rollNo: /^TEST/ })

```
{ acknowledged: true, deletedCount: 20000 }
```

Observation: without the index MongoDB scanned all 20009 documents to find one record. With the index it examined a single entry, so the search cost no longer grows with the size of the collection. This is why rollNo, the field used most often to look up a student, should be indexed.

9.12 Real-time extension: student dashboard

[1] Students scoring above 80

rollNo	name	branch	marks
23CM001	Ravi Kumar	CSE-AIML	90
23CM002	Sneha Reddy	CSE-AIML	92
23CM006	Divya Teja	IT	81
23CM010	Lakshmi Prasad	CSE	93

[2] Students scoring below 50

rollNo	name	branch	marks
23CM005	Karthik Rao	CSE	45

[3] Highest-scoring student

rollNo	name	branch	marks
23CM010	Lakshmi Prasad	CSE	93

[4] Students of branch CSE-AIML

rollNo	name	branch	marks
23CM001	Ravi Kumar	CSE-AIML	90
23CM002	Sneha Reddy	CSE-AIML	92
23CM008	Anitha Devi	CSE-AIML	77

[5] Students sorted according to marks

rollNo	name	branch	marks
23CM010	Lakshmi Prasad	CSE	93
23CM002	Sneha Reddy	CSE-AIML	92
23CM001	Ravi Kumar	CSE-AIML	90
23CM006	Divya Teja	IT	81
23CM008	Anitha Devi	CSE-AIML	77
23CM004	Priya Sharma	ECE	74
23CM003	Arjun Varma	CSE	68
23CM009	Rahul Verma	CSE	58
23CM005	Karthik Rao	CSE	45

[Extra] Branch-wise summary

branch	students	avgMarks	maxMarks
CSE-AIML	3	86.33	92
IT	1	81.00	81
ECE	1	74.00	74
CSE	4	66.00	93

10. Result

The database collegeDB with the students collection was created, and insert, display, search, update, delete and sort operations were performed successfully. An index on rollNo changed the query plan from a collection scan to an index scan, reducing the documents examined from 20009 to 1. The dashboard queries identified the students scoring above 80 and below 50, the highest scorer, the students of a branch and the marks-wise ranking.